



Training Program 1: Advanced NLP & Large Language Models (LLMs)

- M01: Text Preprocessing
- M02: Word and Sentence Embeddings
- M03: Transformer Architectures
- M04: Entity-Centric NLP
- M05: Applied NLP Tasks
- M06: Multimodal NLP
- M07: LLM Fundamentals
- M08: Prompt Engineering
- M09: Fine-Tuning & PEFT
- M10: RAG & Real-World Projects



Advanced NLP & Large Language Models (LLMs)

8.1 Introduction to Prompt Engineering: Why It Matters

8.2 In-Context Learning: Zero-Shot and Few-Shot

8.3 Chain-of-Thought: Eliciting Reasoning and Planning

8.4 Structured Prompting: RISEN and CARE Frameworks

8.5 Summary and Advanced Prompt Strategies

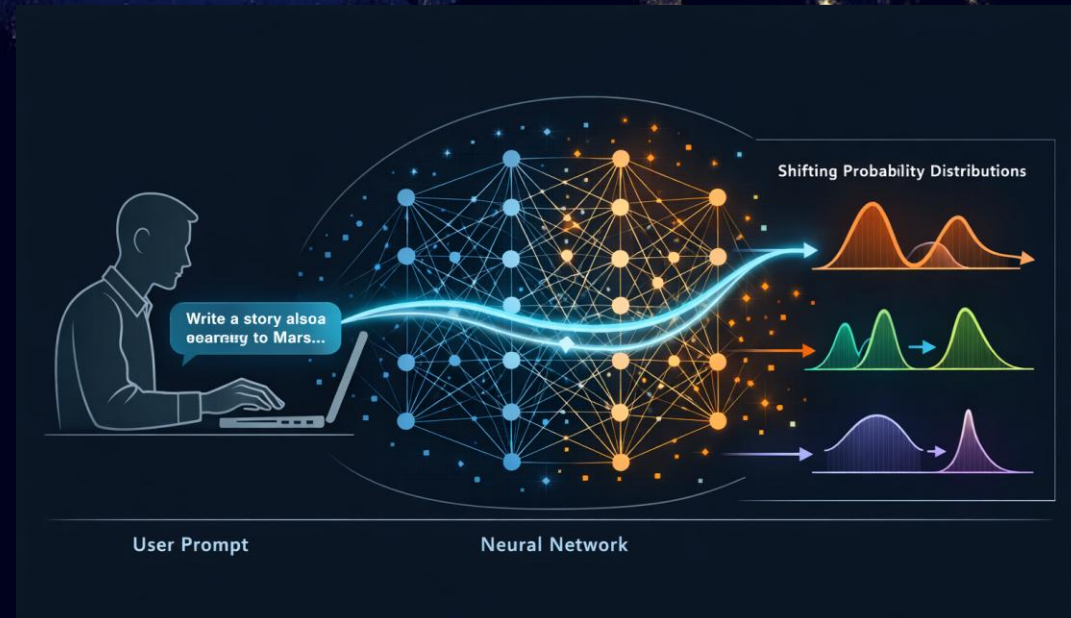
M08 Prompt Engineering

Chapter 8.1 Introduction to Prompt Engineering: Why It Matters

What Is Prompt Engineering?

Prompt Engineering is the primary interface between human intent and large language models. It transforms natural language into a precise control mechanism for probabilistic systems.

- LLMs are controlled through language, not code
- Prompts shape the model's internal probability distribution
- Small wording changes can produce radically different outputs
- Prompting is not chatting, it is system control



M08 Prompt Engineering

Chapter 8.1 Introduction to Prompt Engineering: Why It Matters

Prompting as a Probabilistic Control Mechanism

Every prompt reconfigures the model's internal state by emphasizing specific meanings, roles, and objectives. Effective prompts reduce randomness and increase task precision.

- LLMs predict the next token probabilistically
- Prompts act as soft constraints on attention
- Specificity reduces generic or vague responses
- Role, audience, and intent strongly influence outputs



M08 Prompt Engineering

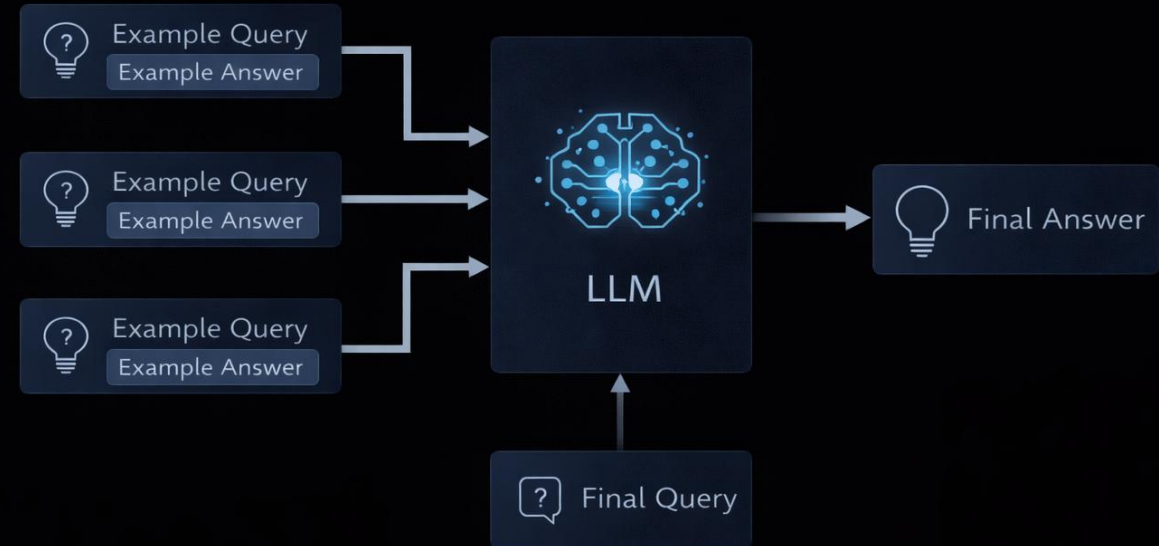
Chapter 8.1 Introduction to Prompt Engineering: Why It Matters

Core Components of a Professional Prompt

Prompt Engineering replaces hard-coded logic with declarative language instructions. Instead of writing rules, developers describe outcomes.

- Prompts function as a new programming layer
- Logic is expressed in natural language
- Focus on what to achieve, not how
- Enables rapid prototyping without training models

Few-Shot Learning



M08 Prompt Engineering

Chapter 8.1 Introduction to Prompt Engineering: Why It Matters

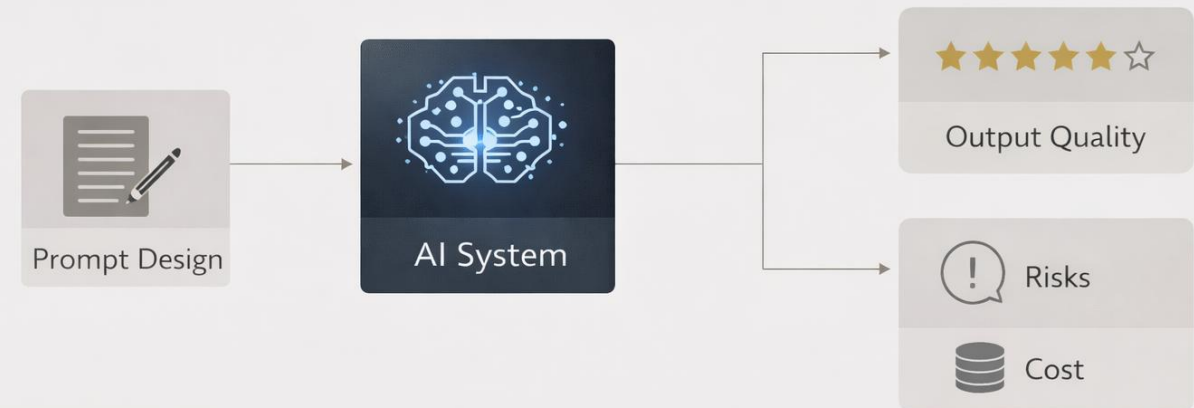
Why Prompt Engineering Matters

Reliable AI systems require structured prompts, not ad-hoc instructions. Professional prompts are designed as modular data objects.

- Instruction: the explicit task
- Context: background knowledge or documents
- Input data: variable user content
- Output constraints: format, tone, structure

Enterprise AI Workflow

Prompt design impacting output quality, risk and cost





Advanced NLP & Large Language Models (LLMs)

8.1 Introduction to Prompt Engineering: Why It Matters

8.2 In-Context Learning: Zero-Shot and Few-Shot

8.3 Chain-of-Thought: Eliciting Reasoning and Planning

8.4 Structured Prompting: RISEN and CARE Frameworks

8.5 Summary and Advanced Prompt Strategies

M08 Prompt Engineering

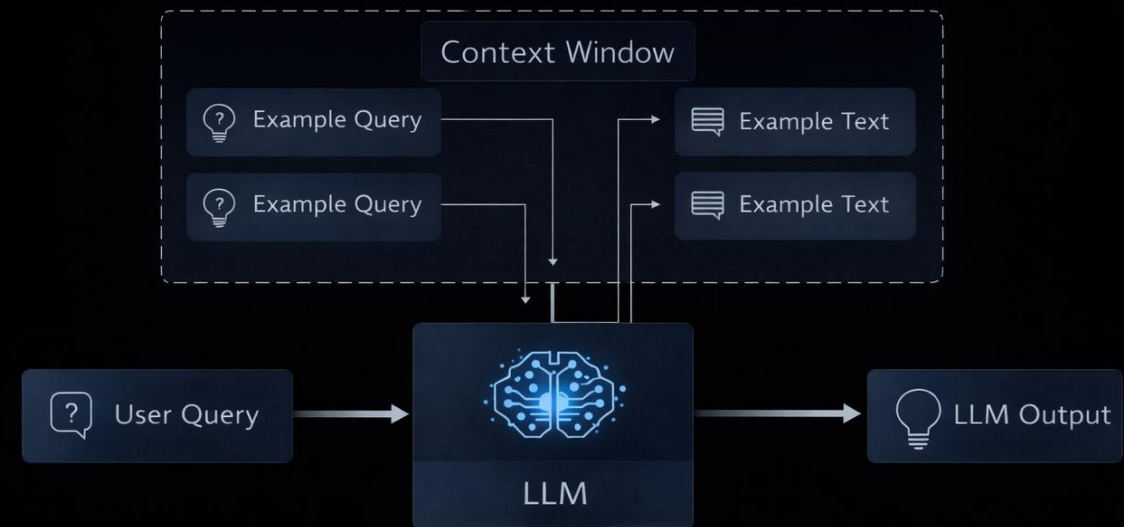
Chapter 8.2 In-Context Learning: Zero-Shot and Few-Shot

What Is In-Context Learning (ICL)?

In-Context Learning allows an LLM to perform new tasks purely from information provided inside the prompt, without updating its weights.

- Learning happens inside the context window
- No gradient updates or retraining involved
- Behavior is guided by prompt structure
- Knowledge is temporary and session-bound

Context Window as Temporary Memory



M08 Prompt Engineering

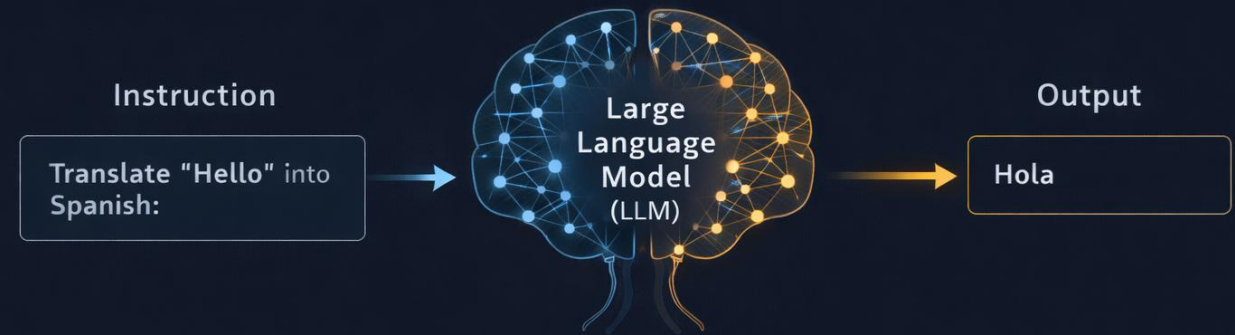
Chapter 8.2 In-Context Learning: Zero-Shot and Few-Shot

Zero-Shot Learning

Zero-shot prompting asks the model to solve a task without seeing any examples, relying entirely on pre-training knowledge.

- No demonstrations provided
- Fast and low token cost
- High flexibility but lower reliability
- Often inconsistent formatting in complex tasks

Zero-Shot



M08 Prompt Engineering

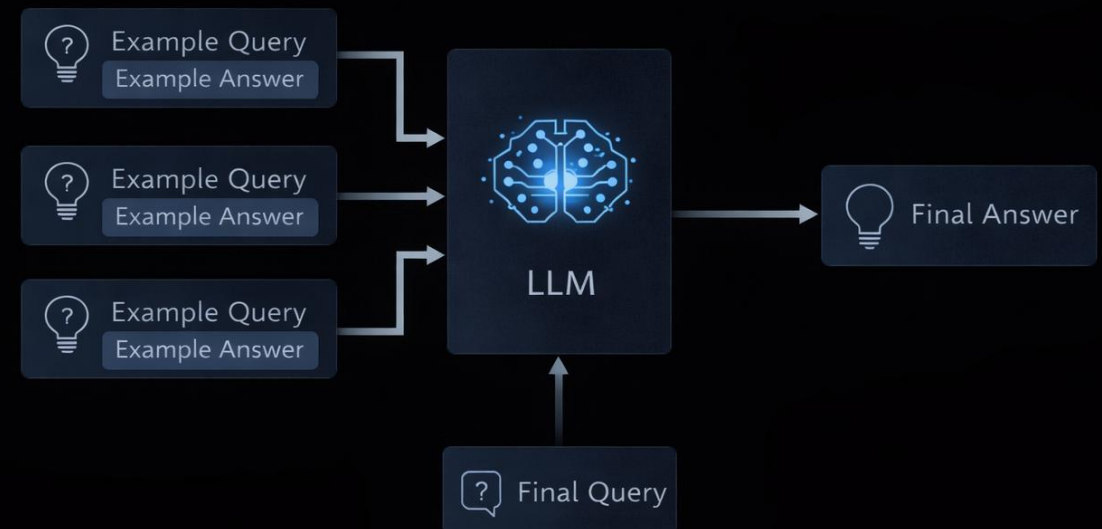
Chapter 8.2 In-Context Learning: Zero-Shot and Few-Shot

Few-Shot Learning

Few-shot learning improves reliability by providing examples that demonstrate the desired input-output behavior.

- Uses 3–10 example pairs
- Examples act as behavioral blueprints
- Strongly improves format and accuracy
- Increases token usage and latency

Few-Shot Learning



M08 Prompt Engineering

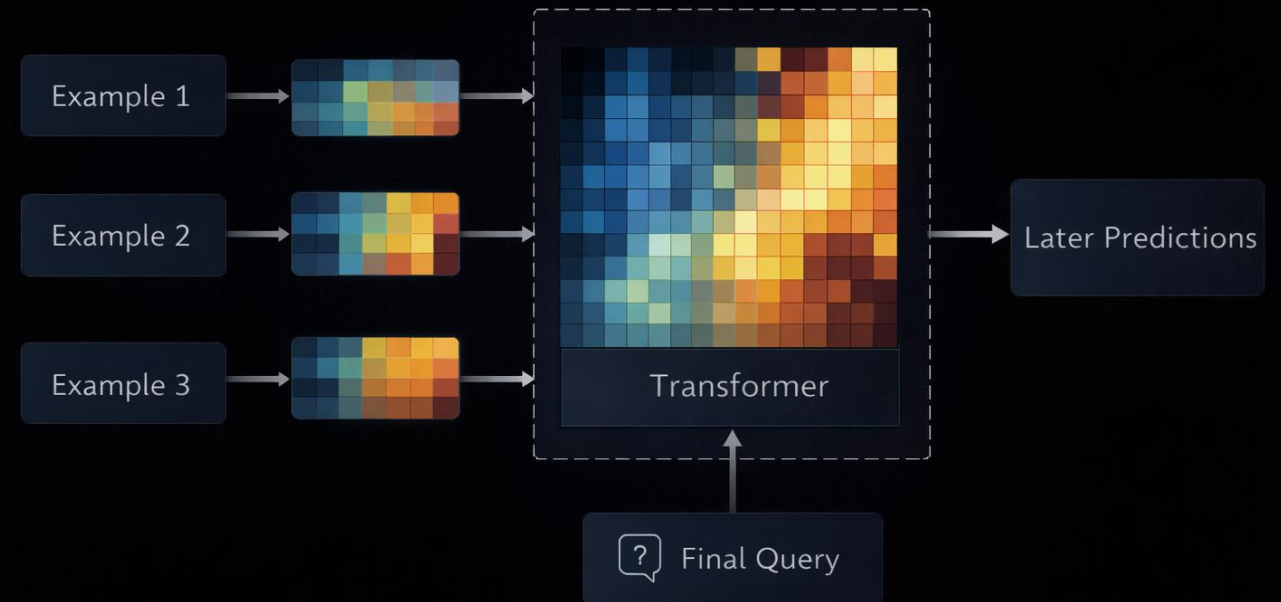
Chapter 8.2 In-Context Learning: Zero-Shot and Few-Shot

Why Few-Shot Works?

Few-shot examples shift the model's internal attention toward a specific task distribution, increasing output probability for correct patterns.

- Primes attention and latent representations
- Reduces ambiguity in task interpretation
- Anchors tone, structure, and labels
- Works best with high-quality examples

Attention Heatmap in Transformer



M08 Prompt Engineering

Chapter 8.2 In-Context Learning: Zero-Shot and Few-Shot

Best Practices for In-Context Exam

Poor example selection can bias or degrade performance. Example quality matters more than quantity.

- Use diverse and representative examples
- Balance labels in classification tasks
- Avoid repetitive patterns
- Order matters due to recency bias

Good Few-Shot Examples	Bad Few-Shot Examples
 Diversity Variety of examples	 Diversity Similar examples only
 Balance Evenly distributed outcomes	 Balance Skewed outcomes
 Order Logical ordering	 Order Random ordering



Advanced NLP & Large Language Models (LLMs)

8.1 Introduction to Prompt Engineering: Why It Matters

8.2 In-Context Learning: Zero-Shot and Few-Shot

8.3 Chain-of-Thought: Eliciting Reasoning and Planning

8.4 Structured Prompting: RISEN and CARE Frameworks

8.5 Summary and Advanced Prompt Strategies

M08 Prompt Engineering

Chapter 8.3 Chain-of-Thought: Eliciting Reasoning and Planning

Why Chain-of-Thought Works?

Standard prompting often fails on complex reasoning because models jump directly to answers. Chain-of-Thought prompting forces the model to externalize intermediate reasoning steps, improving accuracy.

- Encourages step-by-step reasoning
- Reduces logical and arithmetic errors
- Activates latent reasoning capabilities
- Especially effective for math and logic tasks

M08 Prompt Engineering

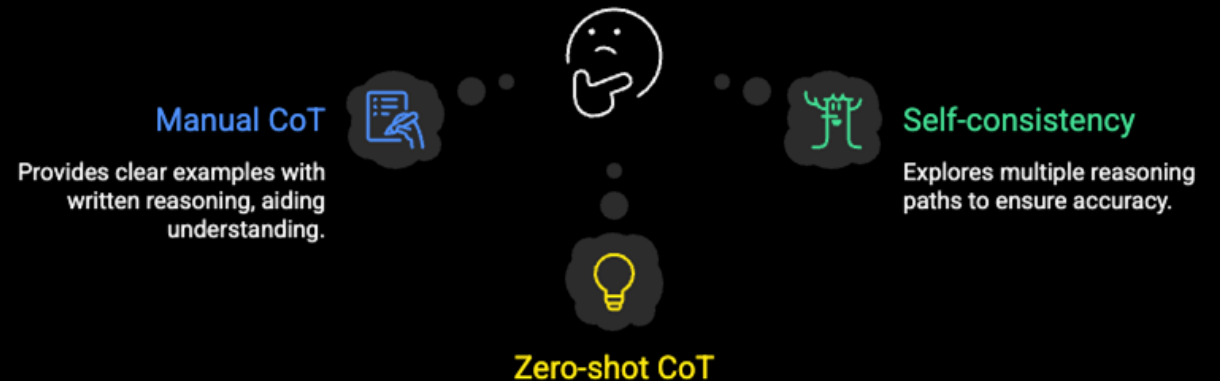
Chapter 8.3 Chain-of-Thought: Eliciting Reasoning and Planning

Types of Chain of Thought

Chain-of-Thought can be manually demonstrated, automatically triggered, or expanded into branching reasoning structures.

- Manual CoT: examples with written reasoning
- Zero-shot CoT: 'Let's think step by step'
- Self-consistency: multiple reasoning paths
- Majority vote filters incorrect logic

Which type of Chain-of-Thought should be used?



M08 Prompt Engineering

Chapter 8.3 Chain-of-Thought: Eliciting Reasoning and Planning

Advanced Reasoning: Tree-of-Thought

Tree-of-Thought extends CoT by exploring multiple reasoning branches, enabling backtracking and deeper planning.

- Branching instead of linear reasoning
- Enables exploration and pruning
- Mimics human problem-solving
- Used for planning and strategy tasks





Advanced NLP & Large Language Models (LLMs)

8.1 Introduction to Prompt Engineering: Why It Matters

8.2 In-Context Learning: Zero-Shot and Few-Shot

8.3 Chain-of-Thought: Eliciting Reasoning and Planning

8.4 Structured Prompting: RISEN and CARE Frameworks

8.5 Summary and Advanced Prompt Strategies

M08 Prompt Engineering

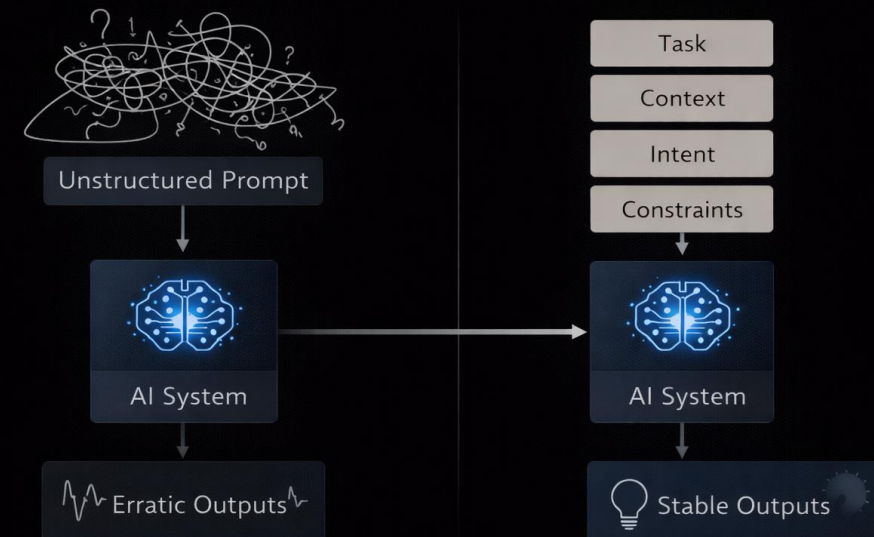
Chapter 8.4 Structured Prompting: RISEN and CARE Frameworks

Why Structured Prompting Matters

Unstructured prompts lead to unstable, inconsistent outputs. Structured prompting frameworks introduce discipline, reproducibility, and reliability into LLM interactions.

- Reduces ambiguity and off-topic responses
- Improves output consistency and format adherence
- Enables scalable, production-grade prompting
- Treats prompts as reusable system components

Chaotic Unstructured Prompt vs Structured Prompt



M08 Prompt Engineering

Chapter 8.4 Structured Prompting: RISEN and CARE Frameworks

RISEN Framework Explained

RISEN provides a comprehensive checklist to control model behavior, reasoning steps, and safety boundaries in professional prompts.

- Role: defines model persona
- Instruction: clear primary task
- Steps: explicit reasoning process
- Examples: quality and format guidance
- Negative constraints: forbidden behaviors

M08 Prompt Engineering

Chapter 8.4 Structured Prompting: RISEN and CARE Frameworks

CARE Framework and Use Cases

CARE is optimized for creative, summarization, and synthesis tasks where tone, context, and output structure are critical.

- Context: background or source material
- Action: what the model must do
- Response: output format and tone
- Examples: style anchoring
- Preferred for writing and analysis tasks

Pros



- ☐ Creative tasks
- ☐ Summarization
- ☐ Synthesis
- ☐ Tone control
- ☐ Contextual understanding

Cons



- ☐ Limited scope
- ☐ Resource intensive
- ☐ Training data

M08 Prompt Engineering

Chapter 8.4 Structured Prompting: RISEN and CARE Frameworks

Production Prompt Architecture

In real systems, prompts are dynamic templates combining fixed system logic with runtime data injection.

- System prompt remains constant
- User data injected dynamically
- Enables millions of safe interactions
- Facilitates testing and versioning

RISEN

- | | |
|---|--|
| R | ☒ Role |
| I | ☒ Instruction |
| S | ☒ Steps |
| E | ☒ Examples |
| N | ☒ Negative Constraints |



Advanced NLP & Large Language Models (LLMs)

8.1 Introduction to Prompt Engineering: Why It Matters

8.2 In-Context Learning: Zero-Shot and Few-Shot

8.3 Chain-of-Thought: Eliciting Reasoning and Planning

8.4 Structured Prompting: RISEN and CARE Frameworks

8.5 Summary and Advanced Prompt Strategies

M08 Prompt Engineering

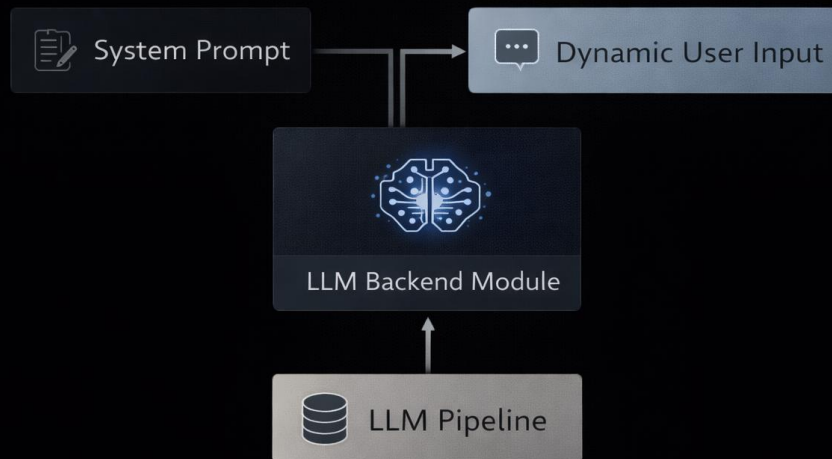
Chapter 8.5 Summary and Advanced Prompt Strategies

Prompt Engineering Best Practices & Security

As LLMs move into production environments, prompt design must prioritize clarity, robustness, and security. Poorly designed prompts increase error rates, cost, and exposure to adversarial manipulation.

- Be explicit and unambiguous in instructions
- Separate instruction, data, and constraints with delimiters
- Always enforce output formats (JSON, tables, schemas)
- Treat prompts as executable system logic
- Defend against prompt injection attacks

System Prompt + Dynamic User Input



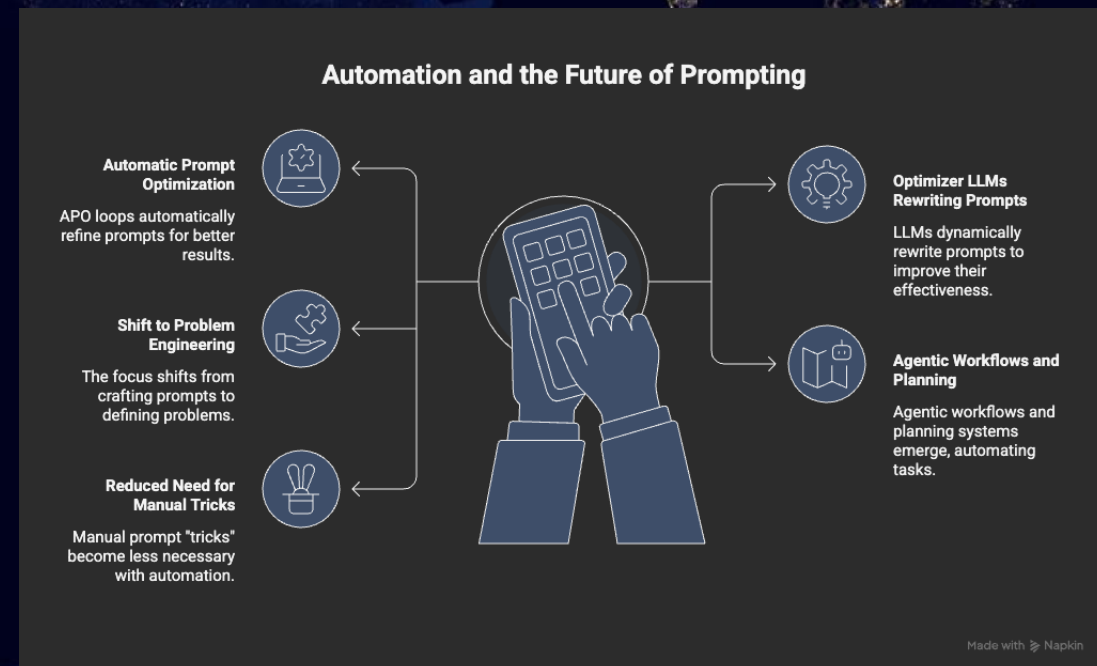
M08 Prompt Engineering

Chapter 8.5 Summary and Advanced Prompt Strategies

Automation and the Future of Prompting

The field is evolving from handcrafted prompts toward automated optimization and agentic systems where prompts become high-level intent specifications.

- Automatic Prompt Optimization (APO) loops
- Optimizer LLMs rewriting prompts dynamically
- Shift from prompt engineering to problem engineering
- Emergence of agentic workflows and planning systems
- Reduced need for manual prompt “tricks”



M08 Prompt Engineering

Key References & Further Reading

- Brown et al. (2020) — Language Models are Few-Shot Learners
- Reynolds & McDonell (2021) — Prompt Programming for Large Language Models
- Liu et al. (2021) — Pre-train, Prompt, and Predict: A Survey of Prompting Methods in NLP
- Wei et al. (2022) — Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
- Wang et al. (2022) — Self-Consistency Improves Chain of Thought Reasoning
- Yao et al. (2023) — Tree of Thoughts: Deliberate Problem Solving with Large Language Models
- Zhou et al. (2023) — Large Language Models Are Human-Level Prompt Engineers
- Greshake et al. (2023) — Compromising Real-World LLM Applications via Indirect Prompt Injection
- Yao et al. (2023) — A Survey on Prompt Engineering for Foundation Models